Each individual test case is now its own describe. Each one uses a let to define the task_string that the creator object is looking for.

The individual assertions are now wrapped in specify calls since the assertion is (arguably) expressive enough not to need another text description. When each assertion references tasks, RSpec calls the let block for tasks, which references creator, lazily triggering that let block—which in turn references task_string, which triggers the let block in that particular test case.

This setup allows each test case to very clearly identify what makes it different from the other cases—the different task_string. In the previous version that information was buried in the noise of the common creation steps. The downside is that the nesting and indirection make it harder to trace execution, especially if you're unfamiliar with this testing style.

Another downside is that the setup is running more frequently, which means the tests will be slower if the setup is doing anything at all complicated.

If you *really* like the single-assertion style, then I recommend the rspec-given gem,[2] which allows you to replace the let and specify blocks with Given, When, and Then. The nicest part of the rspec-given syntax is that you can replace something like specify { expect(tasks.size).to eq(1) } with the somewhat more direct Then { tasks.size == 1 }.

In case you'd like to keep the speed benefits of having multiple assertions in a spec but would also like the benefit of seeing errors past the first one, RSpec offers the :aggregate_failures metadata setting. When applied to any describe or it block, as in RSpec.describe CreatesProject, :aggregate_failures, RSpec won't stop after the first failure, but will show all failures in the spec. (RSpec will still stop on an error.) You can also set :aggregate_failures to be the default in the RSpec configuration, but I don't recommend that, as integration tests, in particular, are kind of annoying and verbose if all their failures are displayed.

Prescription 9	Use :aggregate_failures to get the best features of single-assertion and multiple-assertion tests.

Who Controls the Controller?

All of the new tests pass, so let's take stock. You still have one test pending—your end-to-end test still doesn't like that the create action can't be found in the ProjectsController. Now you have all the pieces you need to write that action.

2. https://github.com/rspec-given/rspec-given